cspm-lite

Fiche d'entretien : ce que le projet fait, et ce qu'il faut savoir expliquer

Synthese a relire avant un entretien technique. Tout ce qui est affirme ici est verifiable dans le depot.

25 controles CIS · 217 tests · 87 % de couverture · 1 dependance runtime.

Genere le 2026-07-30.

Table des matieres

9. Fiche d'entretien — cspm-lite **3**
- 9.1 Les chiffres, vérifiés 3
- 9.2 Ce que le projet fait, en une page 3
- 9.3 Les six décisions à savoir défendre 4
- 9.4 Les six pièges de sécurité implémentés 5
- 9.5 Cinq histoires prêtes à raconter 5
- 9.6 Les questions de relance et leurs réponses 6
- 9.7 Le socle de connaissances à réviser 6
- 9.8 Ce qu'il ne faut pas faire 7
- 9.9 La veille de l'entretien 7

9. Fiche d'entretien — cspm-lite

Ce document est la synthèse à relire avant un entretien. Il répond à deux questions : **ce que le projet fait réellement**, et **ce qu'il faut savoir expliquer**.

Tout ce qui est affirmé ici est vérifiable dans le dépôt. N'affirmez rien d'autre.

9.1 Les chiffres, vérifiés

Élément	Valeur
Contrôles de sécurité	**25** (6 S3, 8 IAM, 6 réseau, 5 CloudTrail), mappés CIS AWS v3.0
Tests	**217**, dont 7 d'intégration avec `moto`
Couverture	**87 %**
Dépendance runtime	**1** — `boto3`
Formats de sortie	4 — JSON, Markdown, HTML, ASFF (AWS Security Hub)
Lignes de code source	~1 540
Qualité	`ruff`, `ruff format`, `mypy --strict` : zéro erreur
Décisions documentées	6 ADR

Fait remarquable à citer : la suite complète tourne aussi avec la seule bibliothèque standard (`python -m unittest`), sans rien installer.

9.2 Ce que le projet fait, en une page

```
API AWS  <??  auth.py          session boto3, profils, AssumeRole + ExternalId
(boto3)         ?
                ?
           collectors/         S3, IAM, EC2/VPC, CloudTrail
           (EXTRACT)           pagination, multi-région parallélisée,
                 ?             AccessDenied géré, erreurs "non configuré" absorbées
                 ?
           Inventory           ??? LA FRONTIÈRE. En amont : boto3.
           (modèle pur)            En aval : zéro dépendance cloud.
                 ?
                 ?
           engine/checks/      25 fonctions pures (Inventory) -> list[Finding]
           engine/registry     registre par décorateur, isolation des pannes
                 ?
                 ?
           config.py           exclusions datées et justifiées
                 ?
                 ?
           reporters/          json · markdown · html · asff
           history.py          SQLite : tendance, diff, MTTR, récidive
                 ?
                 ?
           cli.py              argparse · exit codes 0/1/2 pour la CI
```

Le mode démo : `cspm-lite scan --demo` produit un rapport complet sans compte AWS, sans credentials, sans installation. C'est ce qui rend le projet montrable en 30 secondes.

9.3 Les six décisions à savoir défendre

Chacune a son ADR dans `docs/adr/`. En entretien, annoncez la décision, **puis le compromis** — c'est le compromis qui prouve que vous avez réfléchi.

1. Séparer collecte et évaluation (ADR-001)

Les collecteurs ne connaissent pas la sécurité, les checks ne connaissent pas `boto3`. `Inventory` est la frontière.

Gain : les 25 contrôles se testent en 0,05 s sans réseau ni mock, avec des inventaires fabriqués à la main qui couvrent tous les cas limites. Ajouter un contrôle est une fonction décorée.

Coût assumé : l'inventaire est chargé entièrement en mémoire ; sur un compte de dizaines de milliers de ressources il faudrait passer en flux. Un check ne peut pas déclencher un appel API au vol.

2. Une seule dépendance runtime (ADR-002)

`argparse` plutôt que Typer, `dataclasses` plutôt que pydantic, `tomllib` plutôt que PyYAML, f-strings plutôt que Jinja.

Raison : pour un outil de sécurité, chaque dépendance est une surface d'attaque de la chaîne d'approvisionnement.

Coût : sortie terminal moins jolie, échappement HTML à notre charge — d'où un test dédié à l'injection XSS.

Détail qui porte : TOML plutôt que YAML parce que l'analyseur YAML a un historique de failles d'exécution de code à la désérialisation.

3. Isoler les pannes de checks (ADR-003)

Un check qui lève une exception produit un finding `NOT_AVAILABLE` et le scan continue.

Raison : en production un bucket sera supprimé pendant le scan, un champ d'API sera renommé. Sans isolation, l'utilisateur n'obtient aucun résultat — y compris pour les 24 contrôles qui marchaient.

Subtilité à mentionner : on capture `Exception`, pas `BaseException`. Un Ctrl+C doit interrompre le scan. Un test le vérifie explicitement.

4. Score pondéré par sévérité (ADR-004)

`score = 100 × ?(poids des PASS) / ?(poids des PASS et FAIL)`, avec CRITICAL=10, HIGH=5, MEDIUM=3, LOW=1.

Raison : un taux de conformité brut laisse dix contrôles informatifs compenser un bucket public.

Limite à énoncer d'emblée : les poids sont arbitraires ; le score est comparable dans le temps sur un même compte, pas entre deux comptes de tailles différentes.

5. Exclusions datées et justifiées (ADR-005)

`reason` et `expires_at` sont obligatoires ; leur absence est une erreur fatale. Passée la date, le finding redevient bloquant.

Raison : une exclusion est une acceptation de risque. Sans date, elle devient permanente et personne ne se souvient pourquoi elle existe.

À assumer : c'est plus contraignant que la concurrence. La friction est ici le produit, pas un défaut.

6. Pas d'action GitHub pour les outils de sécurité (ADR-006)

Trivy et Syft tournent depuis leur image officielle épinglée, pas via leur action.

Origine : un vrai échec de CI. `trivy-action`, épinglée par SHA, résout en interne `setup-trivy@v0.2.2` — un tag disparu en amont. Le pipeline est tombé sans qu'une ligne de notre code ait changé.

La leçon, et c'est la meilleure de tout le projet : *l'épinglage par SHA ne protège qu'à la profondeur où on l'applique.* La surface d'attaque réelle est tout l'arbre transitif.

9.4 Les six pièges de sécurité implémentés

Ce sont vos preuves de compréhension du domaine. Chacun est couvert par un test.

Piège	Ce que la plupart des implémentations ratent
`AuthenticatedUsers`	Ne veut pas dire « mes utilisateurs » mais *n'importe quel titulaire d'un compte AWS au monde*. Donc public.
`Statement` dict ou liste	Le langage de policy IAM autorise les deux. Ne traiter que la liste rate la moitié des cas.
`Deny` sur `*`	C'est une protection, pas un risque. Le signaler produit un faux positif. Une `Condition` restreint aussi.
`::/0`	IPv6 aussi ouvert que `0.0.0.0/0`, et systématiquement oublié.
`IpProtocol: "-1"`	Ouvre tous les ports **et** n'a ni `FromPort` ni `ToPort`. Un code qui lit ces champs plante ou conclut à tort.
Clé IAM inactive	Ne produit aucun finding de rotation : ce n'est pas un risque. Le signaler noie le rapport.

Deux subtilités supplémentaires qui montrent la profondeur :

- **Un trail CloudTrail multi-région mais arrêté ne protège rien.** D'où LOG-005, séparé de LOG-001 : `describe_trails` donne la configuration, `get_trail_status` donne l'état réel.
- **UDP 3306 n'est pas une exposition MySQL.** Le protocole compte.

9.5 Cinq histoires prêtes à raconter

Format : situation, ce que j'ai fait, ce que j'en ai tiré. 60 à 90 secondes chacune.

1. La CI cassée par une dépendance transitive. Mon pipeline a échoué en 3 secondes, avant la moindre étape, alors que je n'avais rien changé. La cause : une action que j'avais épinglée par SHA appelait elle-même une sous-action par tag, et ce tag avait disparu. J'ai supprimé l'action au profit de l'image Docker officielle épinglée. J'ai appris que l'épinglage ne vaut qu'à la profondeur où on l'applique. ? *Répond à : « raconte-moi un incident que tu as diagnostiqué. »*

2. Le typage strict qui trouve ce que les tests ne trouvent pas. 217 tests passaient au vert. `mypy --strict` a trouvé 4 erreurs de plus, dont une vraie : une liste typée `list[Any | None]` passée à `str.join`. Le code marchait par chance, parce que le filtre suivait immédiatement. Le typage raisonne sur *tous* les chemins, pas sur ceux auxquels j'ai pensé. ? *Répond à : « à quoi sert le typage si on a des tests ? »*

3. L'identité de finding incomplète. En ajoutant l'historisation, ma clé primaire `(scan, check, ressource)` écrasait silencieusement un finding : le contrôle sur les clés IAM en émet deux par utilisateur, même ARN. J'ai ajouté un champ `subject`. Le bug ne se voyait pas au premier scan et aurait pourri toutes les données trois mois plus tard. ? *Répond à : « une erreur de conception que tu as rattrapée ? »*

4. Le témoin conforme dans le labo. Mon labo Terraform crée des ressources vulnérables, mais aussi un bucket correctement configuré. Un scanner qui met tout en `FAIL` n'est pas plus utile qu'un scanner qui ne détecte rien : le témoin valide l'absence de faux positifs. ? *Répond à : « comment sais-tu que ton outil marche ? »*

5. La friction comme fonctionnalité. J'ai rendu obligatoires la justification et la date d'expiration de toute exclusion. C'est plus contraignant que les outils concurrents, et c'est voulu : sans date, une exception de sécurité devient permanente. J'ai écarté un champ `permanent = true` justement parce que « permanent » est ce qu'on cherche à empêcher. ? *Répond à : « un choix produit que tu as défendu ? »*

9.6 Les questions de relance et leurs réponses

« En quoi c'est différent de Prowler ? » Ce n'est pas différent, c'était volontaire. Prowler est mature, je ne prétends pas faire mieux. L'objectif était de reconstruire l'architecture pour comprendre les décisions qu'elle contient. J'ai lu leur code : eux chargent les checks dynamiquement depuis le système de fichiers, moi j'utilise un registre explicite — plus simple à tracer, moins souple.

« Quelles sont les limites de ton outil ? » *(la meilleure question)* Quatre, énoncées dans le README :

- pas d'**analyse de chemins d'attaque** : je sors des findings isolés, pas le chemin entre une exposition externe et une donnée sensible ;
- pas de **contexte métier**, donc pas de vraie priorisation : un bucket de logs publics est un faux positif que je ne sais pas distinguer ;
- c'est un **instantané**, pas de la détection temps réel ;
- **AWS uniquement**, et mono-compte (pas d'AWS Organizations).

« Pourquoi des threads et pas asyncio ? » Le scan est *I/O bound* : on attend l'API AWS, le GIL est relâché pendant l'attente réseau. `asyncio` aurait demandé `aioboto3` et compliqué le code pour un gain nul à cette échelle — 17 régions, pas 10 000 requêtes.

« Comment testes-tu du code qui appelle AWS ? » Deux niveaux. Les checks sont purs : je leur injecte des inventaires fabriqués, c'est instantané et ça couvre tous les cas limites. Les collecteurs, avec `moto` qui simule AWS en mémoire. Le point important : mon architecture rend le premier niveau possible. C'est le retour sur investissement de la séparation.

« Comment mesures-tu que la posture s'améliore ? » Pas au nombre de vulnérabilités — ce chiffre dépend du périmètre scanné, ajouter une région le fait « empirer » sans qu'aucune configuration ait changé. Je mesure le **délai moyen de correction** par sévérité et le **taux de récidive**, qui mesurent le processus. Techniquement, je distingue l'*état* (le score, lu sur un snapshot) de l'*événement* (l'ouverture et la fermeture d'un finding entre deux scans).

« Comment appliques-tu le moindre privilège ? » Le scanner tourne avec la policy managée `SecurityAudit`, jamais `AdministratorAccess` : un outil d'audit sur-privilégié devient lui-même le risque principal. Le labo Terraform crée un rôle dédié avec `ExternalId`, qui protège du *confused deputy* — sans lui, un autre client du même prestataire pourrait deviner l'ARN du rôle et se faire accéder au compte par le prestataire.

« Qu'est-ce que tu ferais différemment ? » J'aurais figé le modèle de `Finding` plus tôt : je l'ai fait évoluer trois fois, et chaque changement m'a coûté du travail dans les quatre reporters. Et j'aurais écrit le labo Terraform avant les checks, pour valider au fur et à mesure au lieu de découvrir des écarts à la fin.

« Tu l'utiliserais en production ? » Non, pas tel quel. Pas la couverture de Prowler, pas de multi-comptes, pas de retour terrain sur les faux positifs. En revanche il m'a donné les bases pour utiliser et comprendre un vrai CSPM — y compris pour savoir quoi contester dans ses résultats.

9.7 Le socle de connaissances à réviser

À savoir expliquer à l'oral, en moins de deux minutes, sans notes.

Cloud et IAM

- Le modèle de responsabilité partagée : AWS sécurise *le* cloud, le client sécurise *dans* le cloud. Quasiment toutes les brèches médiatisées sont des erreurs client.

- Rôle vs utilisateur IAM : le rôle n'a pas de credentials permanents, on l'assume via STS. C'est toujours la bonne réponse.
- L'ordre d'évaluation des policies : un `Deny` explicite gagne toujours ; sans `Allow` explicite, c'est refusé.
- La chaîne de credentials boto3 : paramètres ? variables d'environnement ? `~/.aws/credentials` ? rôle de conteneur ? rôle d'instance (IMDS).

Les quatre chemins d'exposition d'un bucket S3 ACL, bucket policy, absence de Block Public Access, hébergement de site. Un outil qui n'en vérifie qu'un donne une fausse assurance.

La brèche Capital One (2019) Une SSRF sur une application derrière un WAF mal configuré ? lecture de `169.254.169.254` ? credentials d'un rôle d'instance sur-privilégié ? exfiltration de 100 millions de dossiers depuis S3. 80 M$ d'amende. IMDSv2 corrige la classe de faille en exigeant un token obtenu par `PUT`, ce qu'une SSRF simple ne peut pas produire. C'est le contrôle NET-006.

Réseau Security group = pare-feu *stateful* au niveau de l'interface, autorise seulement. NACL = *stateless* au niveau du sous-réseau, autorise et refuse. L'alternative moderne à l'ouverture du port 22 est **SSM Session Manager**.

Journalisation CloudTrail = plan de contrôle (qui a appelé quelle API). VPC Flow Logs = plan réseau. S3 access logs / data events = plan données. La validation d'intégrité des fichiers CloudTrail permet de détecter *a posteriori* un attaquant qui a effacé ses traces.

Conformité CIS AWS Foundations Benchmark : ~60 contrôles, sections 1.x IAM, 2.x stockage, 3.x logging, 4.x monitoring, 5.x réseau. ASFF est le schéma de Security Hub : produire de l'ASFF, c'est s'insérer dans un flux existant plutôt que d'être ignoré.

DevSecOps Multi-stage : le stage de build a les compilateurs, l'image finale non — moins de surface d'attaque. Non-root : une évasion du processus ne donne pas root. SBOM : l'inventaire des composants, la question posée après Log4Shell. Épinglage par SHA : un tag est mutable —**mais seulement à la profondeur où on l'applique** (ADR-006).

9.8 Ce qu'il ne faut pas faire

1. **Ne présentez pas ce que vous n'avez pas fait tourner.** Si vous n'avez pas encore scanné un vrai compte AWS, dites « le mode démo et les tests d'intégration passent, je déploie le labo cette semaine ».
2. **Ne récitez pas.** Retenez la structure — problème, architecture, validation, apprentissage — pas les phrases.
3. **N'énumérez pas les technologies.** Une décision expliquée vaut dix outils cités.
4. **Ne dites jamais « c'est juste un projet perso ».**
5. **Préparez vos limites autant que vos réussites.** Un candidat sans limites à énoncer n'a pas assez creusé.
6. **Terminez par une question** : « vous utilisez quoi pour la posture cloud, et sur combien de comptes ? »

9.9 La veille de l'entretien

- Relire cette fiche, pas le code.
- Redire le pitch de 2 minutes à voix haute, une fois, chronométré.
- Redessiner l'architecture sur une feuille, sans regarder.
- Choisir **une** des cinq histoires, celle qui colle au poste.
- Vérifier que le dépôt s'ouvre et que le rapport d'exemple s'affiche.